UTN FRLP

Desarrollo de Software 2026

Fundamentos Backend

Clase #14- 13 de julio de 2026

Matías Corti - Martina García Améndola



Lo que quedó corriendo

- En la clase anterior dejamos andando:
  `docker compose up` → **api + db**
- El hello de la api en `http://localhost:3000`
- Modelo de datos verificado + tabla de operaciones del TP (doc **"05 - Diseño de API"**)
- Hoy: **abrimos la caja**
  → qué es ese `api` por dentro, y escribimos nuestro **primer endpoint propio**



Temas de hoy

- Qué es un **servidor** y qué llamamos **backend**
- El ciclo **request → response**
- 🎭 **Simulación:** una API hecha de personas
- **Node.js y npm:** lo que ya usás, del otro lado
- **Express con TypeScript:** el primer endpoint
- 💻 **Práctica:** primer endpoint, todo en Docker
- 📅 Última clase antes del receso (20/07 al 02/08). **Volvemos el 03/08**



¿Quién responde tu fetch?

¿Qué pasa cuando hacés fetch?

- Hoy eso pide… un **archivo estático** (public/libros.json)
- ¿Quién respondería con datos **reales**?
- ¿Dónde viven las **reglas de negocio**?
- ¿Quién decide qué se puede hacer y qué no?

```
// LibrosCatalogo.tsx (C11)

const { data: libros } = useFetch<LibroCardProps[]>('/libros.json');
```



Mocks vs realidad

En tu Librería, **hoy:**

- ¿Quién valida el alta de libro?
  → Zod en el form... **en el navegador de cada uno**
- ¿Qué pasa al recargar la página?
  → los **cambios se pierden** (estado en memoria)
- ¿Quién impide un precio negativo si alguien "esquiva" el formulario?
- ¿Quién guarda los datos para que **todos** vean lo mismo?

ESO que falta es el BACKEND y lo empezamos hoy



Backend

¿Qué es?

El programa del **otro lado del fetch**: recibe pedidos, aplica las reglas y responde con datos.

¿Por qué?

Hay cosas que el front no puede (ni debe) resolver solo

¿Qué hace realmente?

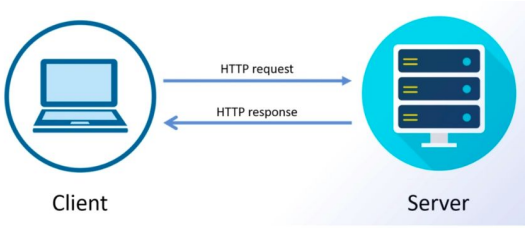
- 🔐 **Valida**: qué se puede hacer
- ⚖️ **Decide**: qué datos mostrar
- 🛡️ **Protege**: quién puede acceder
- 💾 **Persiste**: guarda los datos (DB)

Front = mostrar e interactuar

Back = reglas y datos



Servidor



¿Qué es?
Un **programa** que espera pedidos y los responde. No es (solo) una computadora, es un **proceso "escuchando".**

¿Por qué "cliente-servidor"? (la arquitectura de C03, ahora con código)

- **Cliente** = *quien pide*: el frontend, una app de celular, otro server
- **Servidor** = *quien responde*: tu API
- **Muchos** clientes ↔ **un** servidor
- Ya corriste servidores:
  el dev server de **Vite** lo es 😉

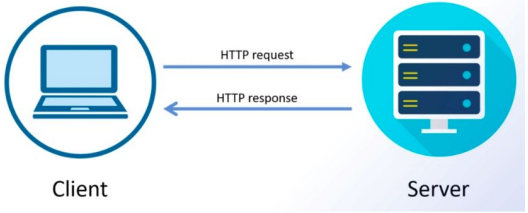


¿Dónde vive un servidor?

Host + Puerto

- Dirección completa = **host : puerto**
- `localhost` = esta misma máquina
- **Puerto** = la "ventanilla":
  un número donde un proceso escucha
- Ya usamos tres:
  - `:5173` (front)
  - `:3000` (api)
  - `:5432` (db)
- El `ports: "3000:3000"` del docker compose mapea la "ventanilla" **de afuera** con la **de adentro** del contenedor

🔄 Request → Response



¿Qué es?
La conversación entre cliente y servidor:
Cliente pide → Servidor procesa → Servidor responde.

Un **Request** dice:

- **Qué** quiero hacer → GET = pedir / POST = enviar
  (ya los usaste: fetch, formularios)
- **Sobre qué** → la ruta (/libros)
- **Con qué datos** → el body (JSON), si hace falta

Un **Response** trae: el **resultado** + los **datos** (JSON)

→ El catálogo completo de verbos y códigos: **C15**

🔁

Mini-cierre: backend y servidores

Lo que nos llevamos:

Concepto	**En una línea**
Backend	El programa detrás del fetch: reglas + datos
Servidor	Proceso que espera pedidos y responde
Cliente	Quién pide (tu React, una app, otro server)
host:puerto	Dónde escucha (localhost:3000)
Request / Response	El pedido y su respuesta



ACTIVIDAD:
SIMULACIÓN API CHAT



Simulación API Chat

Roles y Reglas



NECESITAMOS 11 VOLUNTARIOS
(Uno de cada Grupo)

👥 ROLES

- 📱 5 alumnos = **Apps de chat** (clientes)
  Ana, **B**eto, **C**aro, **D**aniel y **E**uge
- 🔌 2 alumnos = **API Backend** *(procesa la lógica)*
- 🗄️ 1 alumno = **Base de datos** *(guarda mensajes)*
- 📡 3 alumno = **HTTP / Red** *(transporta mensajes)*

🎬 REGLAS DEL JUEGO

- Todo viaja en **papelitos:**
  requests ➡️ y responses ⬅️
- Los roles **no se hablan:**
  solo papelitos, siempre **vía la Red**
- La DB **solo** hace lo que **la API le pide**



Simulación API Chat

Flujos

▶ **Flujo 1: Ana le manda un mensaje a *Beto***

1. 📱 ***Ana*** escribe mensaje para ***Beto***
   → POST /messages *"Hola Beto* 👋*"*
2. 📡 La ***Red*** lo lleva a la ***API***
3. 🔌 La ***API*** valida: ¿*Ana* existe? ¿*Beto* existe? ¿*Ana* puede escribirle *Beto*? ¿*Beto* puede recibir mensajes?
4. 🗄️ La ***API*** le pide a la ***DB***: "guardá esto"
   → la **DB** lo anota en el pizarrón
5. 🔌 La ***API*** arma Response: ✅ "mensaje enviado"
6. 📡 La ***Red*** se la lleva de vuelta a 📱 ***Ana***

🏁 **FIN DEL FLUJO – *Ana*** ve el ✔ en su pantalla...
pero ***Beto*** todavía no sabe nada 👀



Simulación API Chat

Flujos

▶ **Flujo 2: ¿Y cómo se entera *Beto*?**

1. 📱 La app de ***Beto*** consulta cada X segundos
   → `GET /messages?unread=true`
2. 📡 La ***Red*** lo lleva a la ***API***
3. 🔌 La ***API*** valida: *¿Beto* existe? *¿Beto* puede recibir mensajes?
4. 🔌 La ***API*** le pregunta a la ***DB***: "¿hay mensajes sin leer para Beto?"
5. 🗄️ La ***DB*** revisa en el pizarrón:
   → sí, hay uno de *Ana*
6. 🔌 La ***API*** arma Response:
   📩 `[ "Hola Beto 👋" ]`
7. 📡 La ***Red*** se lo lleva a ***Beto***
8. 📱 A ***Beto*** ¡le aparece el mensaje!

🏁 **FIN DEL FLUJO –** El mensaje hizo el viaje completo:
Ana → API → DB → API → Beto



Simulación API Chat

Flujos

▸ **Flujo Aleatorio**

1. Envíense mensaje entre ustedes, uno cada uno a elección.
2. La ***Red*** los trae a la ***API*** y frenamos antes que consulten si hay mensajes nuevos.
3. Se corta internet... ⚡
4. ¡Vuelve! 😅
5. Verifican si hay mensajes nuevos y cerramos el flujo.



Simulación API Chat

Flujos

🎯 **Última Ronda – Ahora resuelven ustedes *(entre todos)***

Caso 1 – 👻 El fantasma

- ***Caro*** le escribe a ***Zoe***... que no existe
- El mensaje llega a la ***API*** y ¿qué pasa?
- 👉 **Ustedes deciden:** ¿qué hace la ***API***? ¿pasa por la ***DB***? ¿qué recibe ***Caro***?

Caso 2 – 🚫 El bloqueado

- ***Euge*** bloqueó a ***Dani***. ***Dani*** le escribe igual...
- 👉 ¿La ***API*** guarda el mensaje? ¿Qué le contesta a ***Dani***? ¿...le dice el motivo real? 🤔

Caso 3 – 👥 El grupo

- ***Euge*** escribe al grupo "**DS2026**", donde están ***Ana***, **Beto** y ***Caro***.
- 👉 ¿Cuántos papelitos manda ***Euge***? ¿Qué guarda la ***DB***? ¿Cómo les llega a los tres?

💡 **Regla de oro:** la ***API*** SIEMPRE contesta algo, aunque sea **"no"**



¿QUÉ DESCUBRIMOS?

🔍 Conclusiones

- ¿Qué rol fue más "**estresante**"?
- ¿Qué pasó cuando "**se cayó internet**"?
- ¿Cómo manejaron **mensajes simultáneos**?
- ¿Qué hubiera pasado sin la base de datos?

💡 En el mundo real

- Los servidores procesan **miles de requests por segundo**
- **Redundancia**: múltiples servidores
- Cada mensaje => **múltiples operaciones** (validar, guardar, avisar...)



Node.js: JavaScript del otro lado



Node.js

¿Qué es?

El **runtime** que ejecuta JS/TS **fuera del navegador** (motor V8, el de Chrome).

¿Por qué?

- ⬅️ Antes: JS solo en navegadores,
  ➡️ Con Node: JS en todas partes
- **Un solo lenguaje** para front y back
  → por eso el stack de la materia
- **Event loop**: un solo hilo que **no se queda esperando** el I/O → por eso existe async/await
- Lo usan Netflix, Uber, LinkedIn, Mercado Libre y muchos otros...



Node vs navegador

Mismo lenguaje, distinto entorno:

	Navegador	**Server Node**
Ejecuta	Solo JavaScript	Solo JavaScript
Ahí corre	Tu app React *(ya convertida a JS)*	Tu API *(ya convertida a JS)*
Tiene	DOM, `window`, `fetch`	Archivos, red, procesos, acceso a DB
No tiene	Acceso a tu disco	DOM ni `window`

Ya usamos Node en el frontend, **Vite corre sobre Node**



NPM Y PACKAGE.JSON

¿Qué es?
El gestor de paquetes de Node (npm) + la ficha del proyecto (package.json).

¿Por qué te suena?
Porque son los mismos de tu front:

- **dependencies**: lo que la app necesita para correr → express
- **devDependencies**: solo para desarrollar → typescript, tsx, @types/*
- El RUN npm install del Dockerfile es **exactamente esto**, dentro de la imagen

Ya hacés (front)	**En backend/ es igual**
npm install	baja dependencias a node_modules/
package.json	scripts + dependencias del proyecto
npm run dev	corre el script "dev"



TypeScript en el back

- El **mismo TS** que escribís para front: interfaces, tipos, todo igual

- Cambia el `tsconfig`: acá `module: "CommonJS"` → el sistema de módulos **nativo de Node** (en el front lo maneja Vite)

- `tsx` = corre TS directo, sin compilar a mano
  `tsx watch` = **recarga sola** al guardar

- `"dev": "tsx watch src/index.ts"` ← el script de `packages.json` que el contenedor ejecuta

🔁

Mini-cierre: Node y npm

Concepto	En una línea
Node.js	Runtime de JS/TS fuera del navegador
Event loop	Un hilo que no espera: I/O asíncrono
npm / package.json	Gestor y ficha del proyecto (igual que en el front)
deps vs devDeps	Para correr vs. para desarrollar
tsx	Corre TS directo; watch recarga sola
CommonJS	El sistema de módulos de Node (tsconfig del back)



Express



Express

¿Qué es?

Un framework **minimalista** para armar APIs sobre Node.

¿Por qué?

- Node solo trae lo básico de red; Express hace fácil lo de todos los días: **rutas, requests, respuestas**
- **Estándar de industria**: Netflix, Mercado Libre, WhatsApp Web...
- Liviano, extensible, y anda perfecto con TypeScript



Anatomía del server mínimo

12 líneas => un servidor real

📁 backend/src/index.ts

```
import express from "express";          // 1. traemos el framework

const app = express();                  // 2. creamos la app
const PORT = 3000;

app.get("/", (_req, res) => {           // 3. definimos un
endpoint
  res.json({ mensaje: "API de la Librería 🐳" });
});

app.listen(PORT, () => {                // 4. escucha pedidos
  console.log(`Servidor en http://localhost: ${PORT}`);
});
```



¿QUÉ ES UN ENDPOINT?

Endpoint = verbo + ruta + handler

```
app.get("/libros", (req, res) => {
  res.json(libros);
});
```

Parte	En el código	Qué hace
Verbo	`get`	qué tipo de pedido atiende
Ruta	`"/libros"`	a qué dirección responde
Handler	`(req, res) => { ... }`	la función que atiende el pedido

El **handler** es una función que **Express** ejecuta cada vez que llega un `GET /libros`:

- Recibe **req** (el pedido que llegó) y **res** (las herramientas para contestar)
- `res.json(datos)` → responde JSON, es lo que recibe en frontend el `useFetch` cuando hace `await res.json()`



Estructura de hoy: un archivo *(a propósito)*

- Hoy: servidor + endpoints + datos, **todo junto**
- Con 2 endpoints se banca, con 20, es un lío
- **Es a propósito**: primero entender el ciclo completo
- Luego iremos planteando estructura más profesional y ordenado: rutas y capas, con nombre y estructura

```
backend/
└── src/
    └── index.ts    ← TODO acá (por ahora)
```

🔁

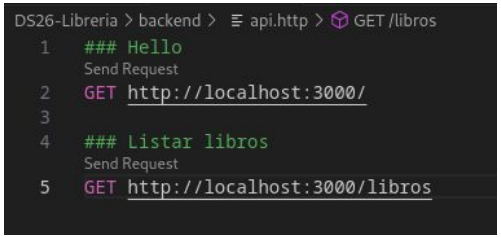
Mini-cierre: Express

Concepto	En una línea
Express	Framework minimalista sobre Node
app.get(ruta, handler)	Define un endpoint
req / res	El pedido / la respuesta
res.json(...)	Contesta JSON (lo que tu fetch recibe)
index.ts único	Maqueta de hoy; C15 la ordena



PRÁCTICA

HERRAMIENTA PARA EL BLOQUE



Para **probar APIs** vamos a usar alguna herramienta que nos permita comunicarnos con ellas.

Opción 1: REST Client es un plugin de VS Code

- **Instalación** (2 min): Extensions (Ctrl+Shift+X) → buscá "REST Client" (autor: Huachao Mao) → Install
- **Uso**: escribís tus requests en un archivo .http que se ejecutan con Send Request y quedan commiteadas en el repositorio

Opción 2: Postman es app aparte

- **Instalación**: se descarga del sitio oficial (https://postman.com/downloads) y se instala
- **Uso**: escribís tus requests en la app y permite guardarlas en ella.



En marcha

Repo de ejemplo:
DS26-Libreria, branch **C14** (está el paso a paso)

Checklist de largada:

- Docker Desktop abierto
- Imagen buildeada de ejercicios/c13-docker/ (si hiciste la actividad)
- REST Client instalado (o tu cliente preferido)
- Bajá lo viejo: docker compose down *(libera el puerto 3000)*



Modo dev en Docker (cómo trabajamos)

La clase pasada insistimos: **la imagen es inmutable**. Sigue siendo cierto ✋

Para desarrollar, le sumamos un **volume de código** al servicio `api`:

- El contenedor **ve tus cambios al instante** → `tsx watch` (script llamado en el CMD) recarga solo
- **Regla:**
  - tocás `src/` → recarga sola
  - tocás lo demás (`package.json`, `tsconfig`, `Dockerfile`) → `docker compose up --build`
- En **producción**: ni volume ni `npm run dev` se usa la imagen cerrada, código compilado adentro.

```
api:
  build: ./backend
  volumes:
    - ./backend/src:/app/src    # ← tu src, montado adentro
```



Hot reload en el contenedor

1. Sumá el volume al `api` de tu `docker-compose.yml` *(snippet ← slide anterior)*

2. `docker compose up`

3. Cambiá el mensaje del hello en `backend/src/index.ts` y guardá

✅ la terminal muestra el reinicio solo y el navegador el texto nuevo, **sin rebuild**

⛑️ ¿No recargó? `docker compose restart api` y seguí *(avisanos al final)*

💻

PRIMER ENDPOINT: GET /LIBROS

En index.ts, con la **misma forma** que el front ya consume de libros.json:

✅ http://localhost:3000/libros devuelve el JSON con tus **≥3 libros**

```
interface Libro {
  id: number;
  titulo: string;
  autor: string;
  precio: number;
  imagen: string;
  disponible: boolean;
}

const libros: Libro[] = [
  { id: 1, titulo: "El principito", autor: "Antoine de Saint-Exupéry",
    precio: 4500, imagen: "https://...", disponible: true },
  // ... 2 libros más → branch C14
];

app.get("/libros", (_req, res) => {
  res.json(libros);
});
```



Probalo como un dev: REST Client

1. Creá el archivo backend/api.http:

2. Click en **Send Request** (aparece arriba de cada request)

3. Mirá la respuesta: **status 200** + el JSON

✅ el 200 y tu array, en el panel de respuesta

```
### Hello
GET http://localhost:3000/

### Listar libros
GET http://localhost:3000/libros
```



El ciclo completo

Tu código nuevo, **empaquetado de verdad:**

- Probá `/libros` de nuevo (navegador o REST Client)
- 👀 Mirá el build: `npm install` salió **de cache** — las capas de C13, en acción

✅ misma respuesta — pero ahora el código vive **dentro de la imagen**

```
docker compose down
docker compose up --build    # nueva imagen, con /libros adentro
```

⚡ ¿Terminaste? Extensiones

- `GET /autores` → hardcodeá 3 autores (misma receta que `/libros`)

- **Extra:** `GET /libros?disponible=true` → filtrá con `req.query` *(como el `?unread=true` de la simulación)*

- Las dos están resueltas en el **branch `C14`** del repo de ejemplo, para comparar después



Lo que falta, falta a propósito

Hoy nos llevamos: **servidor + request/response + Node/npm + Express + primer endpoint corriendo en Docker**

Hueco de hoy	**Lo resuelve**
Todo en un archivo	APIs REST: rutas, capas y orden
Datos hardcodeados	PostgreSQL + Prisma (tu db ya está esperando)
Nadie valida lo que llega	Zod en el back
Cualquiera accede a todo	Auth con JWT
Tu front sigue leyendo el .json	Conexión front ↔ back

Actividades

Actividad Individual

1° instancia hasta el **02/08/2026**
*2° instancia: **13/09/2026***
(Cierre Bloque 3)

[AI-C14] Primer endpoint en tu Librería

1. En ds2026-actividades: copiar el sitio de la librería de la Clase 13 en **ejercicios/c14-backend/**

2. Sumá el **volume de src** (modo dev, como en clase)

3. Creá **GET /libros** con **tus** libros hardcodeados *(la misma forma que tu front ya usa)*

4. Probalo con **REST Client** y con docker compose up --build

5. Commiteá y avisá en #actividades → hilo **"[AI-C14] Primer endpoint"**

Actividad grupal

Hasta el **02/08/2026** (inclusive)

[AG-C14] Backend inicial del TP

1. En el repositorio del TP: creen **backend/** (package.json, tsconfig, src/index.ts, Dockerfile, .dockerignore, .env + .env.example) y el docker-compose.yml en la raíz, tomando como guía: branch **C14** del repositorio de ejemplo

2. Hello + **un GET maqueta de SU dominio**: una entidad de su tabla **"05 - Diseño de API"**, con la forma que su front ya consume

3. PR + review como siempre, aviso en el hilo **"[AG-C14] Backend inicial del TP"**